

GitHub Skills 第 1 节课柴建铭 2023250361021

柴建铭

2026 年 5 月

GitHub Skills 第 1 节课柴建铭 2023250361021

课程名称: Introduction to GitHub

报告主题: GitHub 入门: 仓库、分支、提交与 Pull Request

学生姓名: 柴建铭

学号: 2023250361021

一、核心知识点梳理

本节课主要学习 GitHub 的基础协作流程，重点包括 repository、branch、commit、pull request 和 merge 等概念。Repository 是项目代码和文档的集中存放位置，既可以保存源代码，也可以保存说明文档、图片和配置文件。Branch 用于在不影响 main 分支稳定性的前提下开展新功能或实验性修改；commit 是对一次修改的记录，需要配合清晰的 commit message 说明本次更改的目的；pull request 则是把分支中的修改提交给项目成员讨论、检查和合并的协作入口。

通过本节学习，我理解了 GitHub Flow 的基本思想：先从主分支创建新分支，在分支中完成修改并提交，再通过 pull request 发起合并请求，经过检查后合并到主分支。这个流程能够让多人协作时的修改更加可追踪，也能够通过讨论区、评论和检查记录减少直接修改主分支带来的风险。

二、学习中的难点

学习中的主要难点是区分 Git 与 GitHub 的关系。刚开始容易把 GitHub 理解成单纯的代码存储网站，而忽略 Git 是本地版本控制工具，GitHub 是基于 Git 的远程协作平台。另一个难点是理解分支和提交记录之间的关系：分支不是文件夹，而是指向一系列提交记录的指针。只有理解这一点，才能明白为什么在新分支上修改文件不会立刻影响 main 分支。

在操作层面，pull request 的含义也需要反复理解。它不是简单的“上传作业”，而是一次正式的协作请求，里面包含修改内容、提交记录、对比差异和讨论过程。如果描述不清楚，后续审查者就难以判断修改目的。

三、实践操作与收获

实践过程中，我按照课程提示创建了练习仓库，进入仓库后创建新的分支，在分支中修改 README 文件并提交。提交时我注意填写简洁但明确的提交说明，例如说明本次修改是补充个人介绍或完善文档内容。随后我发起 pull request，查看修改前后的差异，确认没有误改其他内容后完成合并。

本次实践最大的收获是理解了“先分支、后提交、再合并”的规范流程。以前我容易认为只要把文件放到平台上就完成了版本管理，但实际协作中更重要的是让每一次修改都有来源、有说明、有审查记录。这个过程对于后续课程设计、毕业设计或团队项目都有实际意义：当项目出现问题时，可以通过提交记录追溯是谁在什么时候改了什么内容，也可以通过分支把不同功能拆开开发，降低互相影响的概率。

四、学习遇到的困惑与疑问（选做）

目前仍然存在的疑问是：在多人同时开发时，如何判断一个功能应该单独创建一个分支，还是可以直接在已有分支继续修改？如果项目规模较大，分支数量很多，如何制定统一的分支命名规范和合并规则？

五、报告小结

通过本节 GitHub Skills 课程学习，我不仅完成了平台提供的实践任务，也进一步理解了软件工程中“版本控制、协作开发、文档规范、流程管理”的实际意义。后续学习中，我会继续按照规范提交内容，保持报告结构完整、表达准确、实践过程具体，并在项目训练中主动使用 GitHub 记录每一次关键修改。